VHDL Assignment #6: Finite State Machines in VHDL

1 Instructions

- TA in charge: Yousef Safari (yousef.safari@mail.mcgill.ca) - please utilize discussion boards on myCourses for questions as much as possible.
- Due date: Tuesday, December 5, 2023 by 11:59 pm EDT.
- Submission is in teams using myCourses (only one team member submits). In the report, provide the names and McGill IDs of the team members.
- Late submissions will incur penalties as described in the course syllabus.

2 Learning Outcomes

After completing this assignment you should know how to:

- Design FSMs in VHDL
- Design sequence detector circuits
- Test the circuit on the Altera board

3 Introduction

In this VHDL assignment, you will describe a sequence detector in VHDL and test it on the Altera DE1-SoC board using a sequence from a memory input (ROM).

If you need any help regarding the lab materials, you can

- Ask the TA for help during lab sessions and office hours.
- Refer to the text book. In case you are not aware, Appendix A “VHDL Reference" provides detailed information on VHDL.
- You can also refer to the tutorial on Quartus and ModelSim provided by Intel (click here for Quartus and here for ModelSim).
- Refer to the DE1-SoC User Manual (in the Content tab on myCourses).

It is highly recommended that you first try to resolve any issue by yourself (refer to the textbook and/or the multitude of VHDL resources on the Internet). Syntax errors, especially, can be quickly resolved by reading the error message to see exactly where the error occurred and checking the VHDL Reference or examples in the textbook for the correct syntax.

4 Sequence Detector

In this assignment, you will first implement a sequence detector circuit that takes a sequence of bits as its input and detects two different bit patterns in the input sequence. Specifically, you are asked to design a circuit based on Moore-type FSM(s) with an asynchronous reset signal and an enable signal, that takes a sequence of bits as its input “*seq*” and has two outputs “*out_1*” and “*out_2*”. The outputs should be “*out_1* = 1” and “*out_2* = 1” at the clock cycle following the input bit patterns 1011 and 0010, respectively; they should be 0, otherwise. Note that state transitions only occur when the enable signal is high. Otherwise, the FSM stays at its current state. An example of the desired behavior is

clock:	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
seq:	0	0	1	1	0	1	1	0	1	1	0	0	1	0	0	1	0	1	1	0	1
out_1:	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	1	0
out_2:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0

Note: It is best to use two FSMs, each detecting one of the two bit patterns (why?). Your VHDL code will be based on the state diagram(s) of your FSMs. It is, therefore, important that you first come up with a simple state diagram. In fact, you should not need more than five states for each of the two FSMs.

Use the following entity declaration for your VHDL description of the sequence detector:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity firstname_lastname_FSM is
    Port (seq        : in  std_logic;
          enable     : in  std_logic;
          reset      : in  std_logic;
          clk        : in  std_logic;
          out_1      : out std_logic; -- generates 1 when the pattern "1011" is detected;
    otherwise 0.
          out_2      : out std_logic); -- generates 1 when the pattern "0010" is detected;
    otherwise 0.
end firstname_lastname_FSM;
```

`firstname_lastname` in the name of the entity is the name of one of the students in your group.

5 Sequence Counter

In this part, you are required to implement a sequence counter circuit that counts how many times each of the two patterns occurs in the input bit stream. The sequence counter circuit can be realized using the sequence detector circuit, which detects the patterns, followed by two 3-bit up-counters (one for each output of the FSM-based circuit) that keep track of the number of detected patterns. More specifically, each counter is incremented whenever its corresponding pattern has been detected. Use the sequence detector and the 3-bit up-counter from VHDL Assignment #5 to implement the sequence counter circuit with an asynchronous reset and an enable signals.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity firstname_lastname_sequence_detector is
    Port (seq        : in  std_logic;
          enable     : in  std_logic;
          reset      : in  std_logic;
          clk        : in  std_logic;
          cnt_1      : out std_logic_vector(2 downto 0); -- counts the occurrence of the pattern
    "1011".
          cnt_2      : out std_logic_vector(2 downto 0)); -- counts the occurrence of the pattern
    "0010".
end firstname_lastname_sequence_detector;
```

`firstname_lastname` in the name of the entity is the name of one of the students in your group.

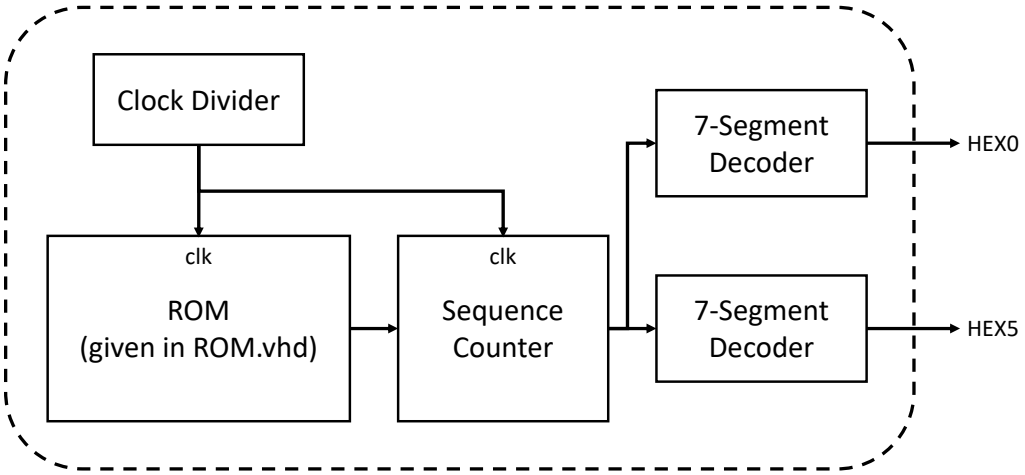
An example of the desired behavior is

clock:	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
seq:	0	0	1	1	0	1	1	0	1	1	0	0	1	0	0	1	0	1	1	0	1
out_1:	0	0	0	0	0	0	0	1	1	1	2	2	2	2	2	2	2	2	2	3	3
out_2:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	2	2	2	2

Note that when the enable signal of the sequence detector circuit is low, its counter and FSM-based instances hold their previous value and state, respectively. Also, the reset signal of the sequence counter should be active low.

6 Demonstration on the FPGA Board

So far, you have designed a circuit counting the number of occurrences of two different patterns within the input sequence of bits. To demonstrate the functionality of your circuit, you must supply it with a sequence of bits and display the number of occurrences of each pattern on the HEX displays. In contrast to previous VHDL assignments where you inserted the inputs using slider switches and push-buttons, the input sequence cannot be inserted in the same fashion. Instead, we store the input bit sequence into a read-only-memory (ROM) and we supply the circuit the bits of the sequence one after the other every 1 second by reading them from the ROM. The ROM circuit takes clock and asynchronous reset signals as inputs, and outputs a bit of the sequence under test at the positive edge of the clock signal when the enable signal is high. The VHDL code for the ROM is given in `ROM.vhd`. To read each bit of the sequence under test every 1 second, you will need to create an instance of the clock divider circuit from VHDL Assignment #5 to enable both the ROM and sequence counter circuits at the appropriate rate (once per second). The 7-segment decoder that you created in VHDL Assignment #4 will be then used to display the occurrence number of each pattern. Note that the *clk* port of all the components (*i.e.*, the clock divider, ROM, and sequence counter circuits) is supplied with a clock frequency of 50 MHz. The following figure shows the high-level architecture of the circuit.



Note that Pushbutton PB0 is used to reset the circuit. When the button is pressed, the circuit has to display 0 on the 7-segment displays until it is released. Recall that the output of a pushbutton is high when the button is not being pushed, and is low when the button is being pushed.

Use the following entity declaration:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity firstname_lastname_wrapper is
    Port (reset     : in  std_logic;
          clk       : in  std_logic;
          HEX0      : out std_logic_vector (6 downto 0),
          HEX5      : out std_logic_vector (6 downto 0));
end firstname_lastname_wrapper;
```

`firstname_lastname` in the name of the entity is the name of one of the students in your group.

Compile the circuit in Quartus. Once you have compiled the circuit, it is time to map it on the Altera DE1-SoC board. Perform the pin assignment for the HEX display, the pushbutton, and the slider switch. Make sure that you connect the clock signal of your design to 50 MHz clock frequency (see the DE1 user's manual for the pin location of 50 MHz clock frequency). Program the board and test the functionality of your circuit.

7 Questions

Please note that even if some of the waveforms may look the same, you still need to include them separately in the report.

1. Why is it better to use two FSMs, rather than one, in the implementation of the sequence detector from Section 4?
2. Briefly explain your VHDL code implementation of all circuits.
3. Provide waveforms for each of the individual circuits (each section) and for the wrapper.
4. Perform timing analysis of the wrapper and find the critical path(s) of the circuit. What is the delay of the critical path(s)?
5. Report the number of pins and logic modules used to fit your design on the FPGA board.

8 Deliverables

You are required to submit the following deliverables on MyCourses. Please note that a single submission is required per group (by one of the group members).

- Lab report. The report should include the following parts: (1) Names and McGill IDs of group members, (2) an executive summary (short description of what you have done in this VHDL assignment), (3) answers to all questions in previous section (if applicable), (4) legible figures (screenshots) of schematics and simulation results, where all inputs, outputs, signals, and axes are marked and visible, (5) an explanation of the results obtained in the assignments (mark important points on the simulation plots), and (6) conclusions. Note - students are encouraged to take the reports seriously, points will be deducted for sloppy submissions. Please also note that even if some of the waveforms may look the same, you still need to include them separately in the report.
- Project files. Create a single .zip file named `VHDL#_firstname_lastname` (replace `#` with the number of the current VHDL assignment and `firstname_lastname` with the name of the submitting group member). The .zip file should include the working directory of the project.